



MobilityAPI

An OGC API – Moving Features server over MobilityDB

A thin compiled tier over MobilityDB · canonical Danish AIS example

net/http + pgxpool · single static binary · no MEOS in the tier



A Thin Tier over MobilityDB, Fat Database

- ▶ **OGC-compliant:** implements OGC API – Moving Features – Part 1: Core (collections, items, temporal geometry, derived queries, temporal properties)
- ▶ **Thin compiled tier:** net/http and a pgx connection pool; a single static binary, no cgo and no MEOS in the process
- ▶ **Fat database:** every temporal computation stays in MobilityDB; the tier never parses or transforms a trajectory in memory
- ▶ **REST for any client:** browsers, mobile, microservices and ETL consume mobility data without SQL or the PostgreSQL wire protocol
- ▶ **Bounded as data grows:** streaming responses, keyset pagination and index-using filters keep memory and paging bounded as collections grow

REQUEST FLOW

Browser

Mobile

ETL / lake

▼ HTTP

OGC API – Moving Features · MF-JSON

MobilityAPI (Go)

net/http + pgxpool

▼ SQL

PostgreSQL wire · pgx

MobilityDB / PostgreSQL

tgeompoint · STBOX · TSTZSPAN

Where the Work Happens

In the tier (Go)

- ▶ **Routing:** net/http with the Go 1.22 pattern mux; one handler per OGC resource
- ▶ **Pooling:** a pgx pool multiplexes requests onto a bounded set of database connections
- ▶ **Adapter:** crs URN to EPSG and interpolation Stepwise to Step, roughly fifty lines
- ▶ **Streaming:** responses are written row by row from a server-side cursor

In the database (MobilityDB)

- ▶ **Reads:** asMFJSON assembles MovingFeaturesJSON in SQL; the tier streams the text
- ▶ **Writes:** tgeompointFromMFJSON parses the payload into a tgeompoint
- ▶ **Filters:** bbox and datetime push to the GiST index; atTime clips
- ▶ **Measures:** speed, cumulativeLength and azimuth compute the derived properties

NO MEOS IN THE TIER

The trajectory is never materialised, parsed or transformed in the tier. It travels as MF-JSON text on the way out and as a MovingFeaturesJSON payload on the way in; MobilityDB does the parsing, the geometry algebra and the temporal computations. The result is a single static binary with no cgo dependency.

MobilityDB SQL vs MobilityAPI REST

PostgreSQL / MobilityDB

```
INSERT INTO ships (id, mmsi, name, trip)
VALUES (205106000, 205106000, 'MARCUS',
       tgeompoint '[
         Point(645829 6058110)@2026-02-26 06:00+00,
         Point(651280 6058390)@2026-02-26 06:25+00,
         Point(656731 6058670)@2026-02-26 06:50+00]'
       );
```

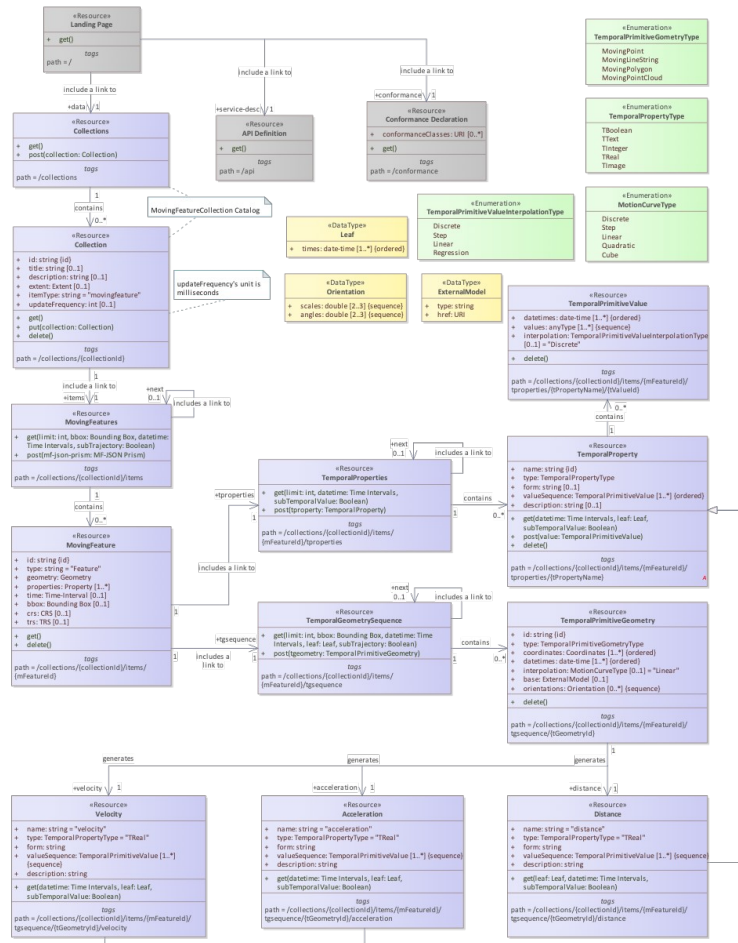
MobilityAPI

```
POST /collections/ships/items
Content-Type: application/json

{
  "type": "Feature",
  "id": "205106000",
  "properties": { "name": "MARCUS" },
  "temporalGeometry": {
    "type": "MovingPoint",
    "datetimes": ["2026-02-26T06:00:00+00", ...],
    "coordinates": [[645829, 6058110], ...],
    "interpolation": "Linear"
  }
}
```

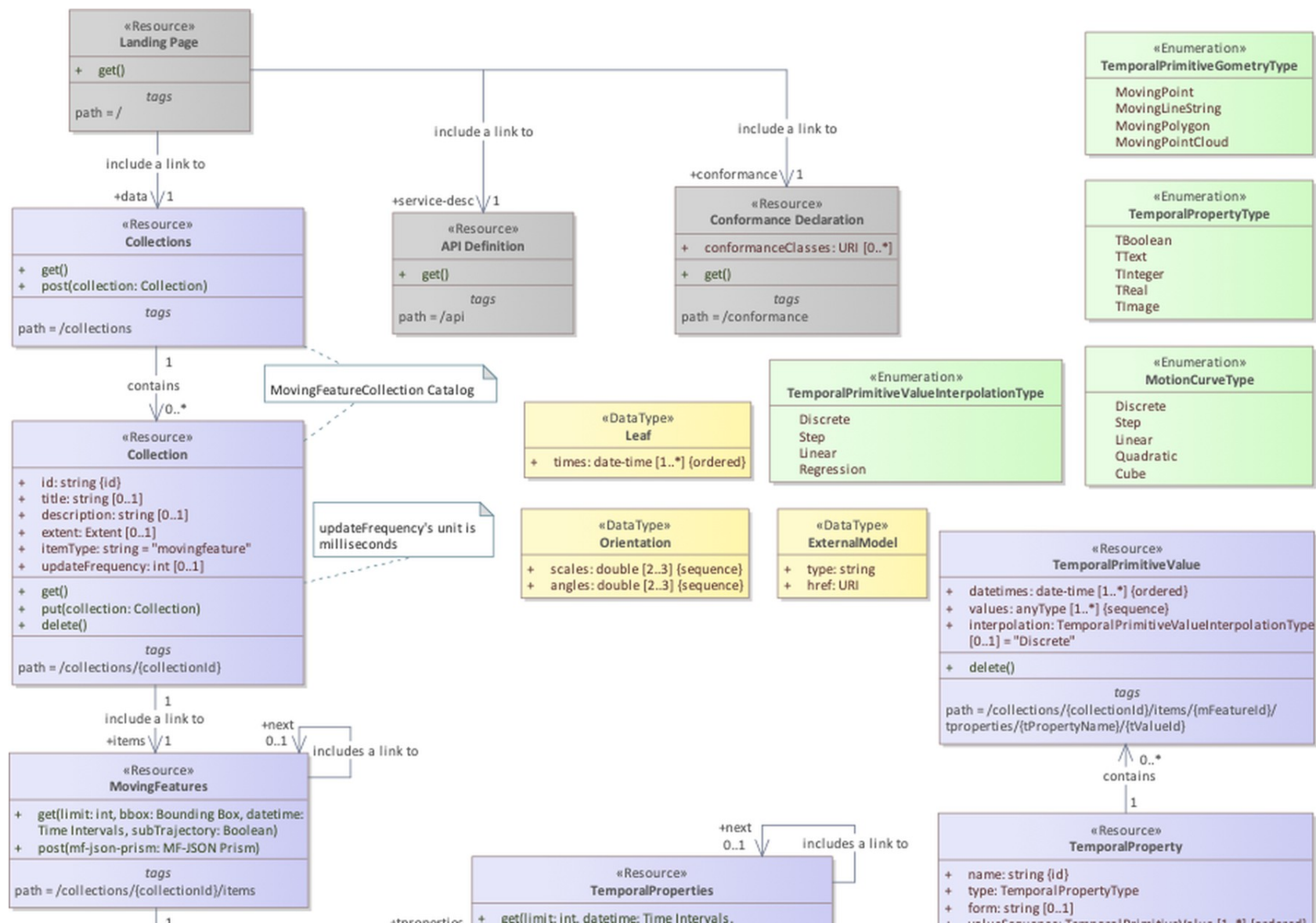
Identical trajectory. The REST client never writes SQL or opens a database connection.

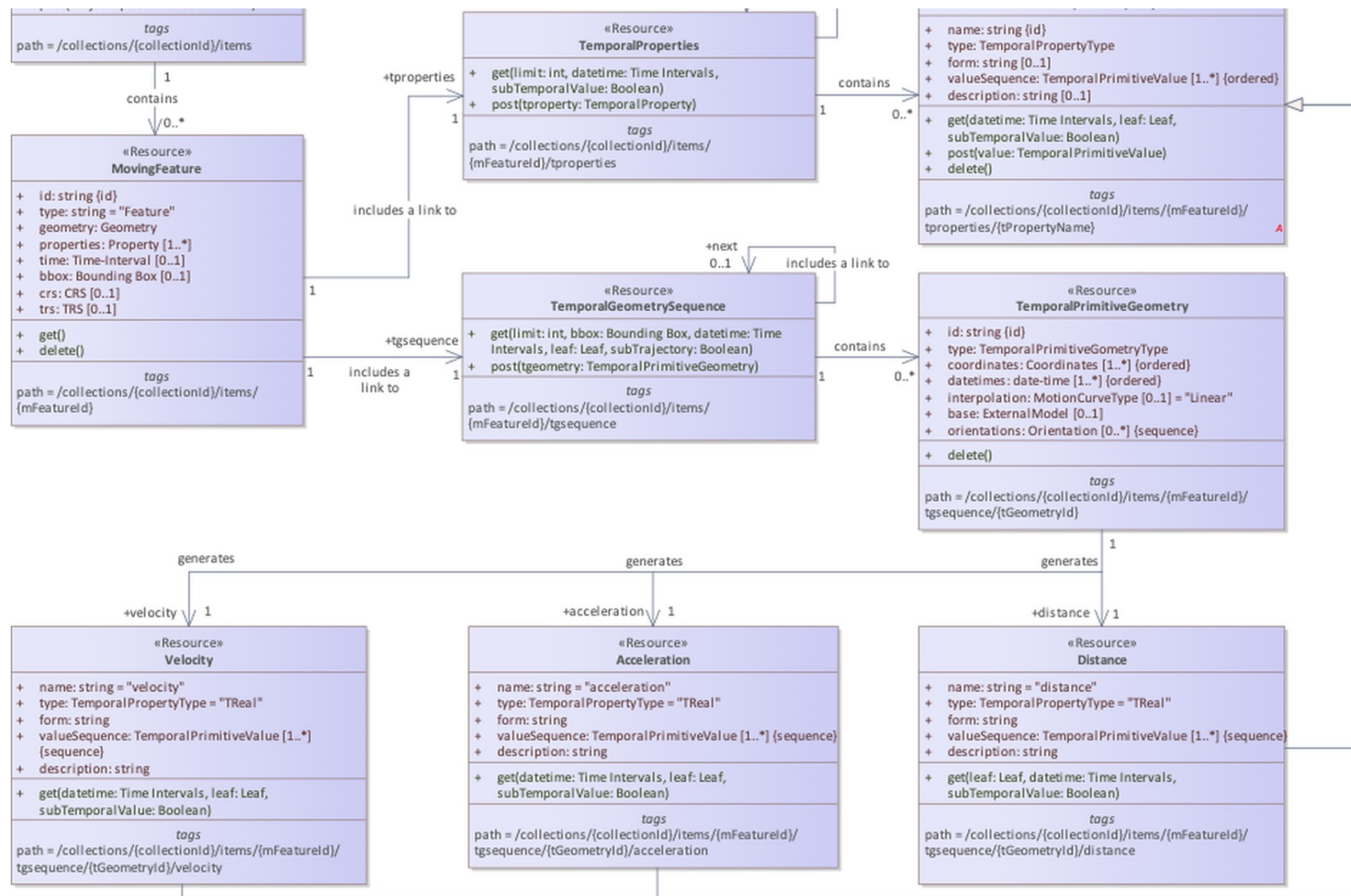
OGC API - Moving Features Data Model



Class diagram — OGC API - Moving Features - Part 1: Core (fig. 1)

- **Collection:** a named group of moving features (itemType movingfeature)
- **MovingFeature:** an item with id, properties, crs and trs
- **TemporalGeometry:** a MovingPoint with parallel coordinates and datetimes plus interpolation
- **Derived queries:** distance and velocity over a temporal geometry
- **TemporalProperties:** time-varying attributes derived from the trajectory





How MobilityAPI Persists Moving Features

collections

PK id	TEXT
title	TEXT
description	TEXT
item_type	TEXT
crs	INTEGER

ships

PK id	INTEGER
mmsi	BIGINT
name	TEXT
trip	tgeompoint

one feature table per collection

- ▶ **collections:** a small registry that maps each collection id to its feature table and CRS
- ▶ **feature table:** one row per moving feature; the trajectory is a single MobilityDB tgeompoint column
- ▶ **GiST index:** gist (trip) backs the bbox and datetime filters with one spatio-temporal index
- ▶ **Derived properties:** speed, distance and heading are computed from trip on demand, not stored

trip is a native MobilityDB type; the bounding box and time span come from its inline STBOX at O(1) cost.

A Day of Danish AIS Ship Traffic

- ▶ **Source:** Danish Maritime Authority AIS feed (aisdata.ais.dk)
- ▶ **Window:** 2026-02-26 to 2026-02-27
- ▶ **2,833 trajectories:** from 1,878 distinct vessels (MMSI), each a MovingPoint
- ▶ **6.2 million instants:** of observed position across the collection
- ▶ **CRS:** EPSG:25832 (ETRS89 / UTM 32N, metres)



Danish AIS vessel trajectories, 2026-02-26

The OGC Endpoint Surface

GET	POST	/collections
GET		/collections/{collectionId}
GET	POST	/collections/{collectionId}/items
GET	DELETE	/collections/{collectionId}/items/{mFeatureId}
GET	POST	/collections/{collectionId}/items/{mFeatureId}/tgsequence
GET		/collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}/distance
GET		/collections/{collectionId}/items/{mFeatureId}/tgsequence/{tGeometryId}/velocity
GET		/collections/{collectionId}/items/{mFeatureId}/tproperties
GET		/collections/{collectionId}/items/{mFeatureId}/tproperties/{tPropertyName}
GET		/ · /api · /conformance

Every path above is OGC API – Moving Features – Part 1: Core. acceleration is offered at the same path but returns 501 (not derivable for this motion model). Items query parameters: bbox, datetime, subtrajectory, leaf, limit, page cursor.

WALKTHROUGH

The Ships Collection, End to End

List → filter → retrieve → derive → write, all over HTTP

List Collections · Retrieve One

GET /collections

200 OK

```
{
  "collections": [
    {
      "id": "ships",
      "title": "ships",
      "itemType": "movingfeature",
      "description": "Danish AIS vessel trajectories",
      "links": [
        { "rel": "items",
          "href": "/collections/ships/items" },
        { "rel": "enclosure",
          "href": "/collections/ships/export",
          "type": "application/x-ndjson" }
      ]
    }
  ]
}
```

GET /collections/ships

200 OK

```
{
  "id": "ships",
  "title": "ships",
  "itemType": "movingfeature",
  "description": "Danish AIS vessel
                trajectories
                (MobilityDB tgeompoint)"
}
```

- ▶ itemType is movingfeature, per OGC API – Moving Features
- ▶ Links are relative; the client follows them without hard-coding paths

Stream the FeatureCollection, Page by Keyset

GET /collections/ships/items?limit=2

```
200 OK Content-Type: application/json
{
  "type": "FeatureCollection",
  "features": [
    { "id": "1", "type": "Feature",
      "properties": { "mmsi": 111219510, "name": "vessel 111219510" },
      "temporalGeometry": {
        "type": "MovingPoint",
        "crs": { "type": "Name",
          "properties": { "name":
"urn:ogc:def:crs:EPSG::25832" } },
        "sequences": [ { "datetimes": ["2026-02-26T13:28:08+01", ...],
          "coordinates": [[721000.4, 6178001.2], ...] } ],
        "interpolation": "Linear" } },
    { "id": "2", "type": "Feature", ... }
  ],
  "numberReturned": 2,
  "links": [ { "rel": "next",
    "href": "/collections/ships/items?after=2&limit=2" } ]
}
```

- ▶ **Streamed:** each Feature is written from a server-side cursor; memory is bounded for any page size
- ▶ **Keyset next:** the next link advances by id (after=), with no OFFSET
- ▶ **asMFJSON:** the temporalGeometry is assembled in SQL and streamed as text
- ▶ **limit:** caps the page; the tier enforces a maximum

Retrieve a Single Vessel

GET /collections/ships/items/1

```
200 OK
{
  "id": "1",
  "type": "Feature",
  "properties": {
    "mmsi": 111219510,
    "name": "vessel 111219510"
  },
  "temporalGeometry": {
    "type": "MovingPoint",
    "crs": { "type": "Name", "properties": {
      "name": "urn:ogc:def:crs:EPSG::25832" } },
    "sequences": [
      { "datetimes": ["2026-02-26T13:28:08+01", ...],
        "coordinates": [[721000.4, 6178001.2], ...] } ],
    "interpolation": "Linear"
  }
}
```

- ▶ **One movement:** the full trajectory of one vessel as MovingFeaturesJSON
- ▶ **crs:** carried as an OGC URN; the tier maps to and from the EPSG form
- ▶ **interpolation:** Linear, so the vessel moves continuously between fixes
- ▶ **sequences:** one per continuous run; gaps split the trajectory

Vessels Intersecting a Bounding Box

GET

```
/collections/ships/items?  
bbox=720000,6160000,738000,6178000
```

```
GET /collections/ships/items  
?bbox=720000,6160000,738000,6178000  
&limit=10000
```

```
// bbox = minx,miny,maxx,maxy in the storage CRS  
// EPSG:25832 (metres)  
// 170 of 2,833 trajectories cross the box
```

- **bbox:** keeps vessels whose trajectory intersects the envelope
- **Index:** pushed to the MobilityDB GiST index via the overlap operator
- **CRS:** coordinates are in the collection's storage CRS (EPSG:25832)
- **Composable:** with datetime, subtrajectory, limit and after



Trajectories crossing the box (teal) over other traffic (grey); the box in red

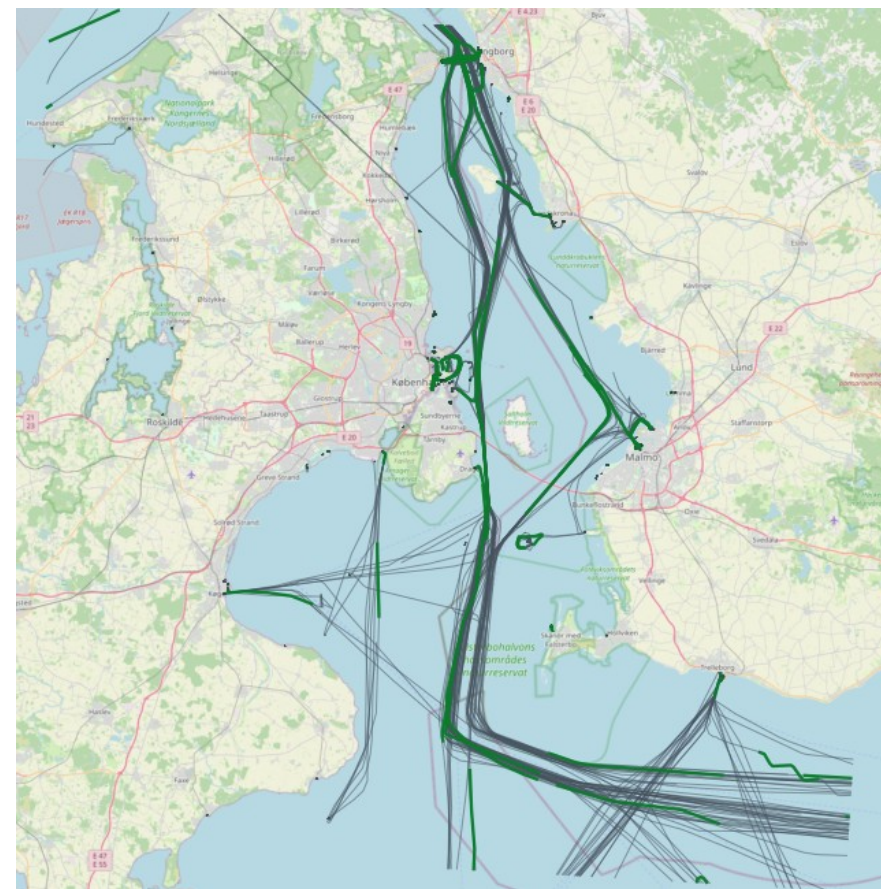
Restrict Trajectories to a Time Interval

GET /collections/ships/items?subtrajectory=true

```
GET /collections/ships/items
  ?subtrajectory=true
  &datetime=2026-02-26T14:00:00+01
    /2026-02-26T15:00:00+01

// datetime is an RFC 3339 interval (start/end)
// subtrajectory=true clips each trajectory to the
// window with atTime, instead of filtering trips
```

- ▶ **subtrajectory:** clips each trajectory to the datetime interval via atTime
- ▶ **datetime:** RFC 3339 interval; a bare instant is also accepted
- ▶ **Gap aware:** rows whose values fall entirely in a temporal gap are dropped
- ▶ **Validation:** subtrajectory requires a bounded interval



Subtrajectories within a one-hour window (green) over full trajectories (grey)

Create · Append · Delete

POST /collections/ships/items

```
{ "type": "Feature", "id": "900001",
  "properties": { "name": "demo" },
  "temporalGeometry": {
    "type": "MovingPoint", "interpolation": "Linear",
    "datetimes": ["2026-02-26T10:00:00+01", ...],
    "coordinates": [[600000, 6330000], ...] } }
```

```
201 { "message": "created", "id": "900001" }
```

POST /collections/ships/items/900001/tgsequence

```
// append a temporally-disjoint sub-trajectory
200 { "message": "appended", "id": "900001" }

// overlapping the existing time span:
409 { "code": "409",
      "description": "the sub-trajectory overlaps
                     the existing temporal geometry in time" }
```

DELETE /collections/ships/items/900001

204 remove the feature

- **merge:** append uses MobilityDB merge, which rejects time overlap; the tier maps that to 409
- **Derived properties:** are computed from the geometry, so writes to them return 501; modify the geometry instead
- **Single geometry:** a feature carries one temporal geometry; delete the feature rather than a primitive

The Temporal-Geometry Sequence

GET /collections/ships/items/1/tgsequence

```
200 OK
{
  "type": "MovingPoint",
  "crs": { "type": "Name", "properties": {
    "name": "urn:ogc:def:crs:EPSG::25832" } },
  "sequences": [
    { "datetimes": ["2026-02-26T13:28:08+01", ...],
      "coordinates": [[721000.4, 6178001.2], ...],
      "lower_inc": true, "upper_inc": true }
  ],
  "interpolation": "Linear"
}
```

Selectors on this resource

- **datetime:** clip the sequence to an interval (atTime)
- **leaf:** return the geometry at one or more instants
- **POST:** append a temporally-disjoint sub-trajectory

The temporal geometry is the trajectory itself, returned as MovingFeaturesJSON straight from asMFJSON.

Distance · Velocity · Acceleration

GET /collections/ships/items/1/tgsequence/0/velocity

```
200 OK
{ "name": "velocity", "type": "TReal", "form": "m/s",
  "description": "Speed over ground ...",
  "valueSequence": [
    { "datetimes": ["2026-02-26T13:28:08+01", ...],
      "values": [22.78, 11.03, ...],
      "interpolation": "Stepwise" } ],
  "links": [ { "rel": "self", ... } ] }
```

- ▶ **velocity:** speed over ground from speed(trip), piecewise-constant (Stepwise)
- ▶ **distance:** cumulative length from cumulativeLength(trip) (Linear)
- ▶ **acceleration:** not fabricated, because with linear position the speed is a step function whose derivative is not a finite function
- ▶ **Exact:** each measure is a MobilityDB function reshaped into an OGC temporalProperty (TReal)

GET .../tgsequence/0/acceleration

```
501
{ "code": "501", "description":
  "acceleration is not derivable: linearly
  interpolated position gives a piecewise-constant
  speed, whose derivative is zero within each
  segment and undefined at the vertices" }
```

Time-Varying Attributes per Feature

GET /collections/ships/items/1/tproperties

```
200 OK
{
  "temporalProperties": [
    { "name": "velocity", "type": "TReal", "form": "m/s" },
    { "name": "distance", "type": "TReal", "form": "m" },
    { "name": "heading", "type": "TReal", "form": "rad" }
  ],
  "numberReturned": 3,
  "links": [ { "rel": "self", ... } ]
}
```

GET /collections/ships/items/1/tproperties/velocity?
leaf=...

```
{ "name": "velocity", "type": "TReal", "form": "m/s",
  "valueSequence": [
    { "datetimes": ["2026-02-26T13:28:08+01"],
      "values": [22.78],
      "interpolation": "Discrete" } ] }
```

Property type

TBoolean · TText · TInteger · TReal · TImage

- ▶ **Derived:** velocity, distance and heading are computed from the trajectory, read-only
- ▶ **valueSequence:** datetimes and values with each segment's true interpolation
- ▶ **leaf:** selects values at given instants; the result is Discrete
- ▶ **datetime:** clips the property to an interval

Declared Classes, API Definition, Errors

Conformance classes

- ▶ movingfeatures / conf / common
- ▶ / conf / mf-collection
- ▶ / conf / movingfeatures
- ▶ ogcapi-common-1 / conf / core
- ▶ ogcapi-common-2 / conf / collections

API definition

- ▶ **/api:** an OpenAPI 3.0 definition
- ▶ **service-desc:** linked from the landing page
- ▶ **Landing:** self, service-desc, conformance, data
- ▶ Clients discover the surface at runtime

OGC error responses

- ▶ **400:** invalid parameters or body
- ▶ **404:** collection / feature not found
- ▶ **409:** sub-trajectory overlaps in time
- ▶ **501:** measure not derivable / derived write
- ▶ Body: { "code", "description" }

Bounded Memory at Any Result Size

393 MB

in one streamed response

≈18 MB

resident memory, bounded by the stream
and not the result size

- ▶ **Streaming responses:** the FeatureCollection is written row by row from a server-side cursor, so memory is bounded for any result size
- ▶ **Keyset pagination:** next links advance by id with no OFFSET, so paging stays constant cost through arbitrarily large collections
- ▶ **Index-using filters:** bbox and datetime push to the MobilityDB GiST index; subtrajectory clips with atTime

Every temporal computation stays in MobilityDB; the tier is a single static binary. Streaming, keyset paging and index-using filters keep cost bounded as the database grows.

Extensions Beyond the Standard

Not part of OGC API - Moving Features; excluded from conformsTo.

GET /collections/{collectionId}/export

- ▶ **NDJSON (default):** one Feature per line, streamed; DuckDB, Spark and pandas ingest it directly
- ▶ **format=parquet:** columnar trajectory WKB plus a bbox and time sidecar; the row group bounds export memory
- ▶ **enclosure link:** advertised from the collection via the IANA enclosure relation

393 MB

exported at about

18 MB

resident, bounded by the row group

PUT /collections/{collectionId}/items/{mFeatureId}

- ▶ **In-place replace:** the standard allows only GET and DELETE on a feature; PUT replaces its geometry and properties

A step toward the Data Lake House architecture in the MobilityDB ecosystem.

One Tier, Many MEOS Engines

Thin OGC API - Moving Features tier

MF-JSON / WKB wire format · streaming · keyset paging · index pushdown · no MEOS in the tier

▼ serves any MEOS-backed engine; the engine becomes a configuration choice

MobilityDB

PostgreSQL

Transactional serving over a live database.

Implemented today

MobilityDuck

DuckDB

Embedded, Parquet-native lakehouse queries.

Direction

MobilitySpark

Apache Spark

Distributed batch over very large histories.

Direction

MobilityFlink

Apache Flink

Streaming, real-time moving features.

Direction

MEOS types and the MF-JSON / WKB wire format are shared across the ecosystem, so the same tier fronts a transactional database today and the lakehouse and streaming engines as they mature.

The Standard Surface, Recapped

COLLECTIONS

- ✓ list (GET)
- ✓ retrieve (GET)
- ✓ registry-backed

MOVING FEATURES

- ✓ insert (POST)
- ✓ retrieve / stream
- ✓ delete (DELETE)
- ✓ keyset paging

QUERYING

- ✓ bbox spatial filter
- ✓ datetime interval / instant
- ✓ subtrajectory clipping
- ✓ leaf selector

TEMPORAL GEOMETRY

- ✓ get sequence (GET)
- ✓ append sub-trajectory (POST)
- ✓ datetime / leaf clip

DERIVED MEASURES

- ✓ distance
- ✓ velocity
- ✓ acceleration → 501 (not derivable)
- ✓ exact, as TReal

CONFORMANCE & SCALE

- ✓ 5 conformance classes
- ✓ OpenAPI at /api
- ✓ streaming + keyset
- ✓ index-using filters

Thanks for Listening

Questions?



Find Out More

■ **OGC API – Moving Features – Part 1: Core**
<https://docs.ogc.org/is/22-003r3/22-003r3.html>

■ **MobilityDB**
<https://github.com/MobilityDB/MobilityDB> · <https://mobilitydb.com>

■ **MEOS / ecosystem overview**
<https://libmeos.org>

■ **AIS data — Danish Maritime Authority**
<http://aisdata.ais.dk>